

Face Special Effects

Face Special Effects

1. Course Content
2. Program Startup
3. Core Code Analysis

1. Course Content

- Run example programs using the dlib library to implement face detection and face special effects functionality.

2. Program Startup

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

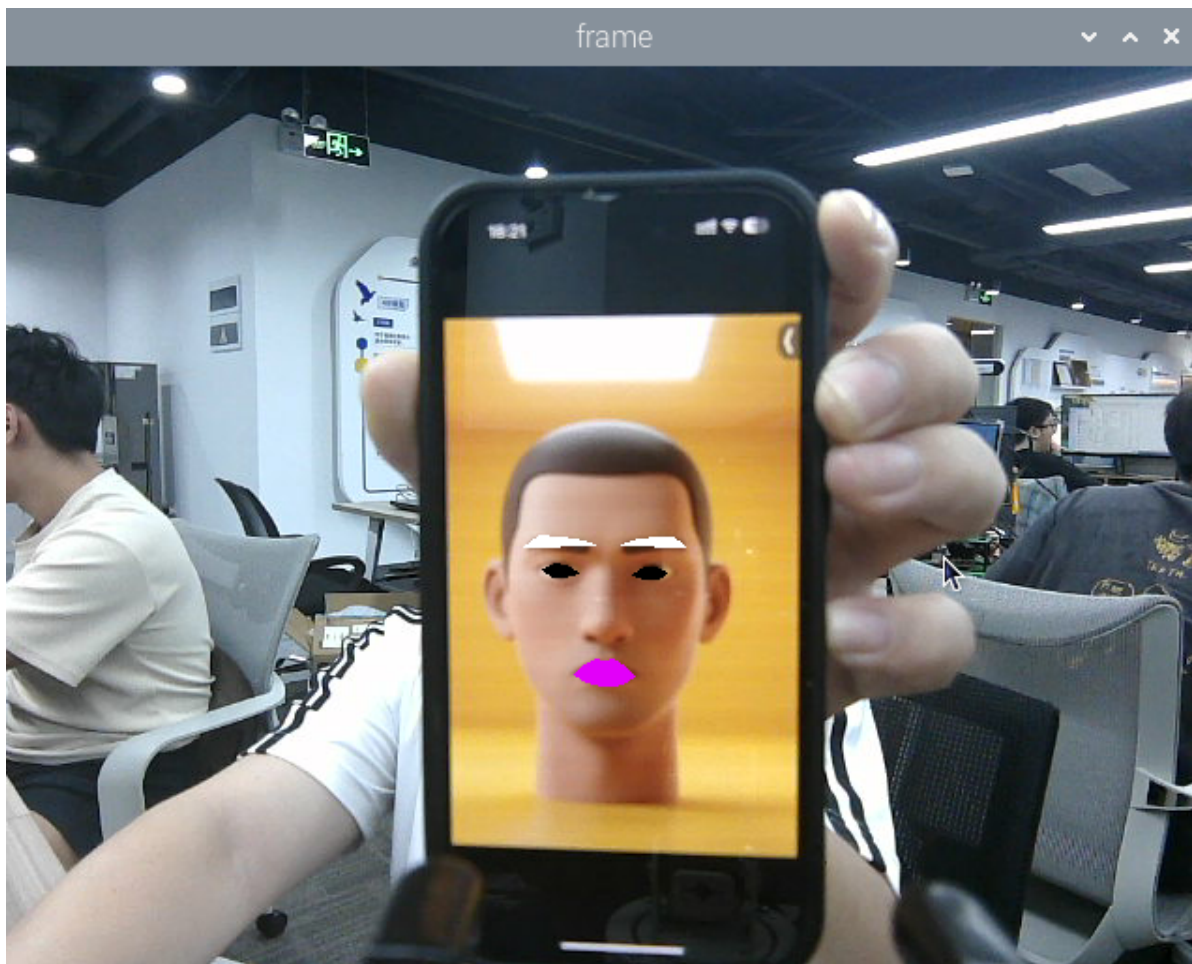
Open a terminal inside the container,

```
exec_m3
```

- Run face special effects node in terminal

```
ros2 launch m3_bringup mediapipe_demo.launch.py demo:=FaceLandmarks
```

After the program runs, as shown in the figure below, it will first detect faces, then apply special effects to the eyebrows, eyes, and mouth areas,



3. Core Code Analysis

Program code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/mediapipe/06_FaceLandmarks.py
```

Import the required library files,

```
import time
#import dlib library
import dlib
import cv2 as cv
import numpy as np
import rclpy
from rclpy.node import Node
from cv_bridge import CvBridge
from sensor_msgs.msg import Image
from arm_msgs.msg import ArmJoints
import cv2
```

dlib library introduction:

DLIB is a modern C++ toolkit containing machine learning algorithms and tools for creating complex software in C++ to solve real-world problems. It is widely used in industry and academia in fields such as robotics, embedded devices, mobile phones, and large high-performance computing environments. The dlib library uses 68-point landmarks to mark important parts of the

face, for example, points 18-22 mark the right eyebrow, and 51-68 mark the mouth. The dlib library's `get_frontal_face_detector` module is used to detect faces, and the `shape_predictor_68_face_landmarks.dat` feature data is used to predict face feature values.

The 68-point facial landmarks of dlib are arranged in the following order:

- 0-16: Jaw contour
- 17-21: Right eyebrow
- 22-26: Left eyebrow
- 27-35: Nose bridge and tip
- 36-41: Right eye
- 42-47: Left eye
- 48-67: Lip contour

Initialize data and define publishers and subscribers,

```
def __init__(self, name):
    super().__init__(name)
    #Parameter declaration and acquisition

    self.declare_parameter('rgb_topic', '/ascamera_hp60c/camera_publisher/rgb0/image')

    rgb_topic=self.get_parameter("rgb_topic").get_parameter_value().string_value
    self.dat_file = os.path.join(os.path.expanduser('~'),
    'MODELS', 'mediapipe', 'shape_predictor_68_face_landmarks.dat')
    self.hog_face_detector = dlib.get_frontal_face_detector()

    self.dlib_facelandmark = dlib.shape_predictor(self.dat_file)

    self.rgb_bridge = CvBridge()

    self.sub_rgb = self.create_subscription(
        Image, rgb_topic, self.get_RGBImageCallback, 5)
```

Color image topic callback function,

```
def get_RGBImageCallback(self, msg):
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(msg, "bgr8")
    #Pass the obtained image to the defined get_face function and return the
    detected image
    frame = self.get_face(rgb_image, draw=False)
    #Call the prettify_face function to apply special effects to the image
    frame = self.prettify_face(frame, eye=True, lips=True, eyebrow=True,
    draw=True)
    key = cv2.waitKey(1)
    cv.imshow('frame', frame)
```

get_face function, detect faces,

```
def get_face(self, frame, draw=True):
    #Convert image space, convert bgr to grayscale for subsequent image
    processing
    gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
    #Pass grayscale image to detect faces
    self.faces = self.hog_face_detector(gray)
```

```

for face in self.faces:
    self.face_landmarks = self.dlib_facelandmark(gray, face)
    if draw:
        for n in range(68):
            x = self.face_landmarks.part(n).x
            y = self.face_landmarks.part(n).y
            cv.circle(frame, (x, y), 2, (0, 255, 255), 2)
            cv.putText(frame, str(n), (x, y), cv.FONT_HERSHEY_SIMPLEX, 0.6,
(0, 255, 255), 2)
        return frame

```

prettify_face function, add special effects to the face,

```

def prettify_face(self, frame, eye=True, lips=True, eyebrow=True, draw=True):
    #Eyes
    if eye:
        leftEye = self.get_lmList(frame, 36, 42)
        rightEye = self.get_lmList(frame, 42, 48)
        if draw:
            if len(leftEye) != 0: frame = cv.fillConvexPoly(frame,
np.mat(leftEye), (0, 0, 0))
            if len(rightEye) != 0: frame = cv.fillConvexPoly(frame,
np.mat(rightEye), (0, 0, 0))
    #Lips
    if lips:
        lipIndexlistA = [51, 52, 53, 54, 64, 63, 62]
        lipIndexlistB = [48, 49, 50, 51, 62, 61, 60]
        lipsUpA = self.get_lipList(frame, lipIndexlistA, draw=True)
        lipsUpB = self.get_lipList(frame, lipIndexlistB, draw=True)
        lipIndexlistA = [57, 58, 59, 48, 67, 66]
        lipIndexlistB = [54, 55, 56, 57, 66, 65, 64]
        lipsDownA = self.get_lipList(frame, lipIndexlistA, draw=True)
        lipsDownB = self.get_lipList(frame, lipIndexlistB, draw=True)
        if draw:
            if len(lipsUpA) != 0: frame = cv.fillConvexPoly(frame,
np.mat(lipsUpA), (249, 0, 226))
            if len(lipsUpB) != 0: frame = cv.fillConvexPoly(frame,
np.mat(lipsUpB), (249, 0, 226))
            if len(lipsDownA) != 0: frame = cv.fillConvexPoly(frame,
np.mat(lipsDownA), (249, 0, 226))
            if len(lipsDownB) != 0: frame = cv.fillConvexPoly(frame,
np.mat(lipsDownB), (249, 0, 226))
    #Eyebrows
    if eyebrow:
        lefteyebrow = self.get_lmList(frame, 17, 22)
        righteyebrow = self.get_lmList(frame, 22, 27)
        if draw:
            if len(lefteyebrow) != 0: frame = cv.fillConvexPoly(frame,
np.mat(lefteyebrow), (255, 255, 255))
            if len(righteyebrow) != 0: frame = cv.fillConvexPoly(frame,
np.mat(righteyebrow), (255, 255, 255))
    return frame

```

get_lmList function to get face coordinates,

```

def get_lmList(self, frame, p1, p2, draw=True):

```

```

#Define an empty list
lmList = []
#Check if face is detected
if len(self.faces) != 0:
    #Iterate through the face list to get the xy coordinates of points
    within each interval
    for n in range(p1, p2):
        x = self.face_landmarks.part(n).x
        y = self.face_landmarks.part(n).y
        #Add the point coordinates on the face to the list
        lmList.append([x, y])
        if draw:
            next_point = n + 1
            if n == p2 - 1: next_point = p1
            x2 = self.face_landmarks.part(next_point).x
            y2 = self.face_landmarks.part(next_point).y
            cv.line(frame, (x, y), (x2, y2), (0, 255, 0), 1)
    return lmList

```